Scaling and Updating Applications

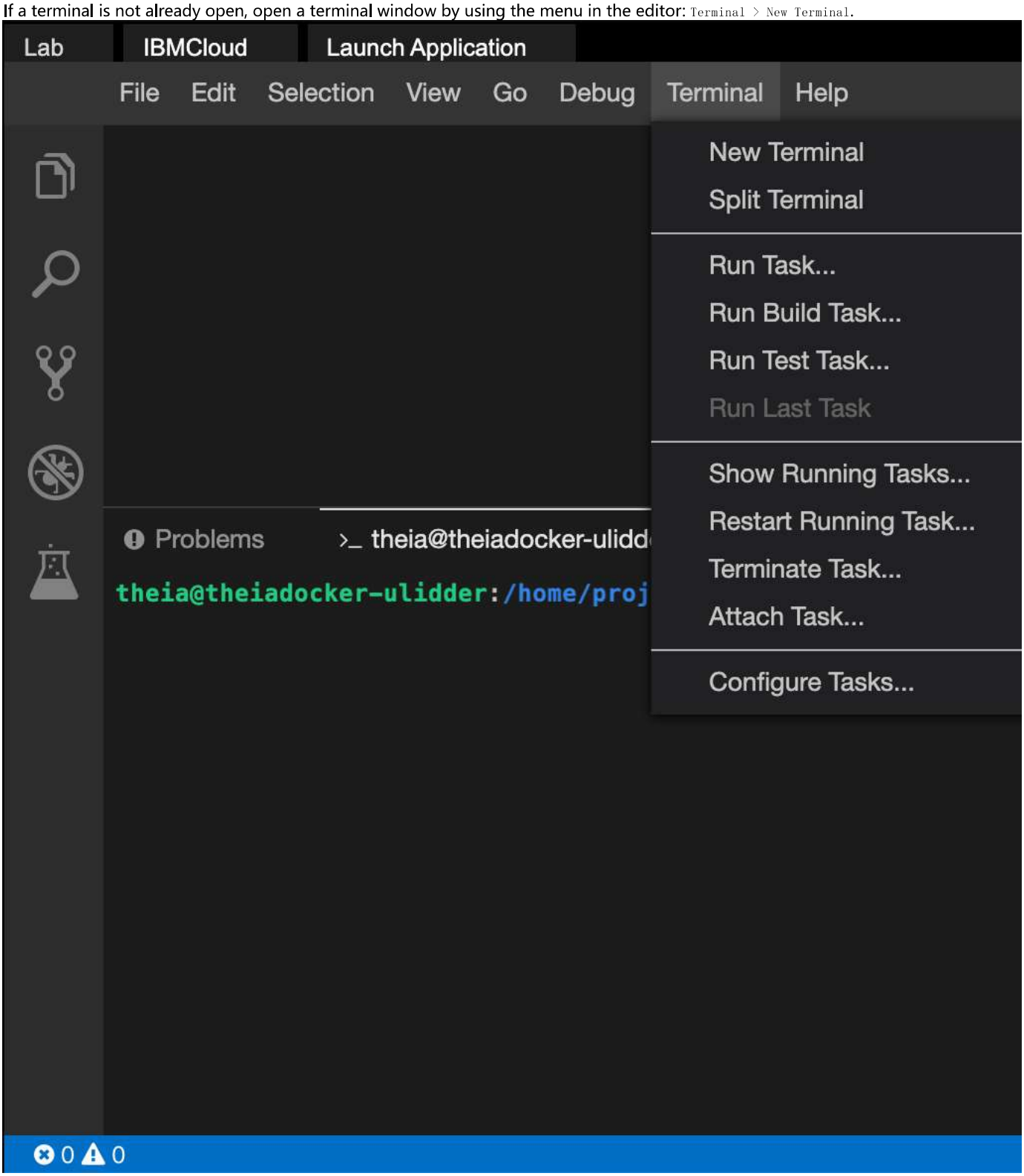
Objectives

In this lab, you will:

- Scale an application with a ReplicaSet
- Apply rolling updates to an application
- Use a ConfigMap to store application configuration
- Autoscale the application using Horizontal Pod Autoscaler

Verify the environment and command line tools

1. If a terminal is not already open, open a terminal window by using the menu in the editor: `Terminal > New Terminal`.



NOTE: It might take sometime for the Termainal Prompt to appear. In case you are unable to see the terminal prompt even after 5 minutes, please close the browser tab and relaunch the lab again.

2. Change to your project folder.

NOTE: If you are already in the `/home/project` please skip this step.

```
1
cd /home/project
```

Copied!

3. Clone the git repository that contains the artifacts needed for this lab, if it doesn't already exist.

```
1
[ ! -d 'CC201' ] && git clone https://github.com/ibm-developer-skills-network/CC201.git
```

Copied!

```
theia@theiadocker-[redacted]:/home/project$ [ ! -d 'CC201' ] && git clone https://github.com/ibm-develop
er-skills-network/CC201.git
Cloning into 'CC201'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 20 (delta 6), reused 19 (delta 6), pack-reused 0
Unpacking objects: 100% (20/20), done.
```

4. Change to the directory for this lab.

```
1
cd CC201/labs/3_K8sScaleAndUpdate/
```

Copied!

```
theia@theiadocker-[redacted]:/home/project$ cd CC201/labs/3_K8sScaleAndUpdate/
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

5. List the contents of this directory to see the artifacts for this lab.

```
1
ls
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ ls
app.js  deployment-configmap-env-var.yaml  deployment.yaml  Dockerfile  package.json
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

Build and push application image to IBM Cloud Container Registry

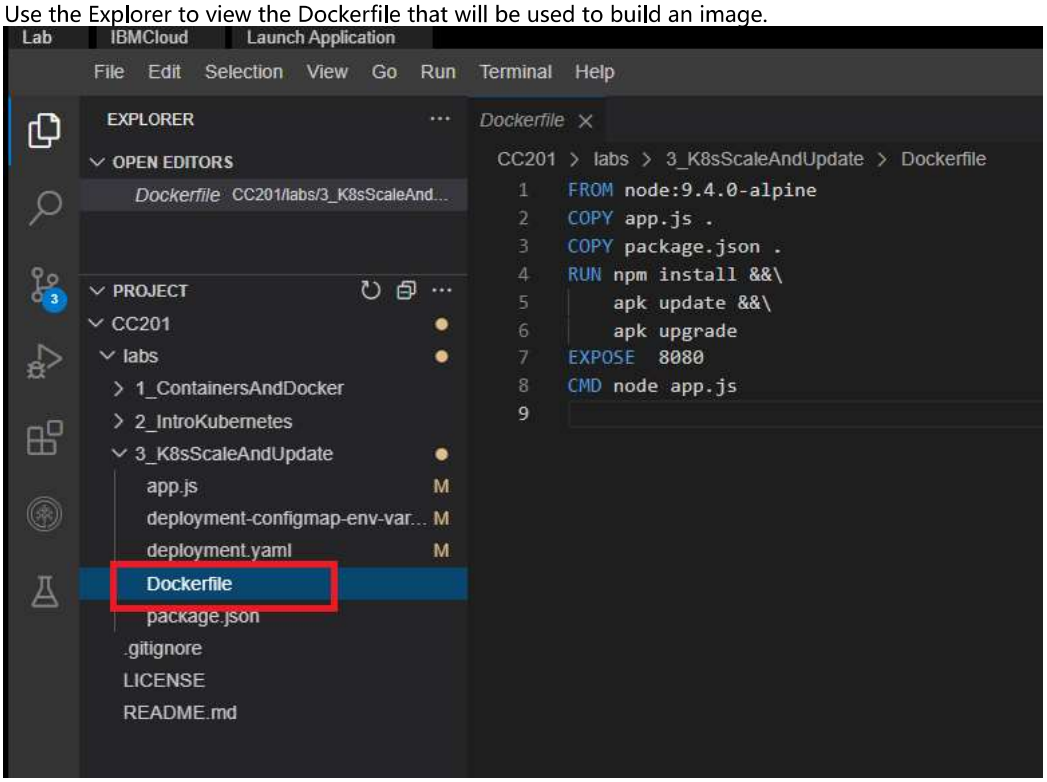
1. Export your namespace as an environment variable so that it can be used in subsequent commands.

```
1
export MY_NAMESPACE=sn-labs-$USERNAME
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ export MY_NAMESPACE=sn-labs-$USERN
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

2. Use the Explorer to view the Dockerfile that will be used to build an image.



3. Build and push the image again, as it may have been deleted automatically since you completed the first lab.

```
1
```

1. `docker build -t us.icr.io/$MY_NAMESPACE/hello-world:1 . && docker push us.icr.io/$MY_NAMESPACE/hello-world:1`

Copied!

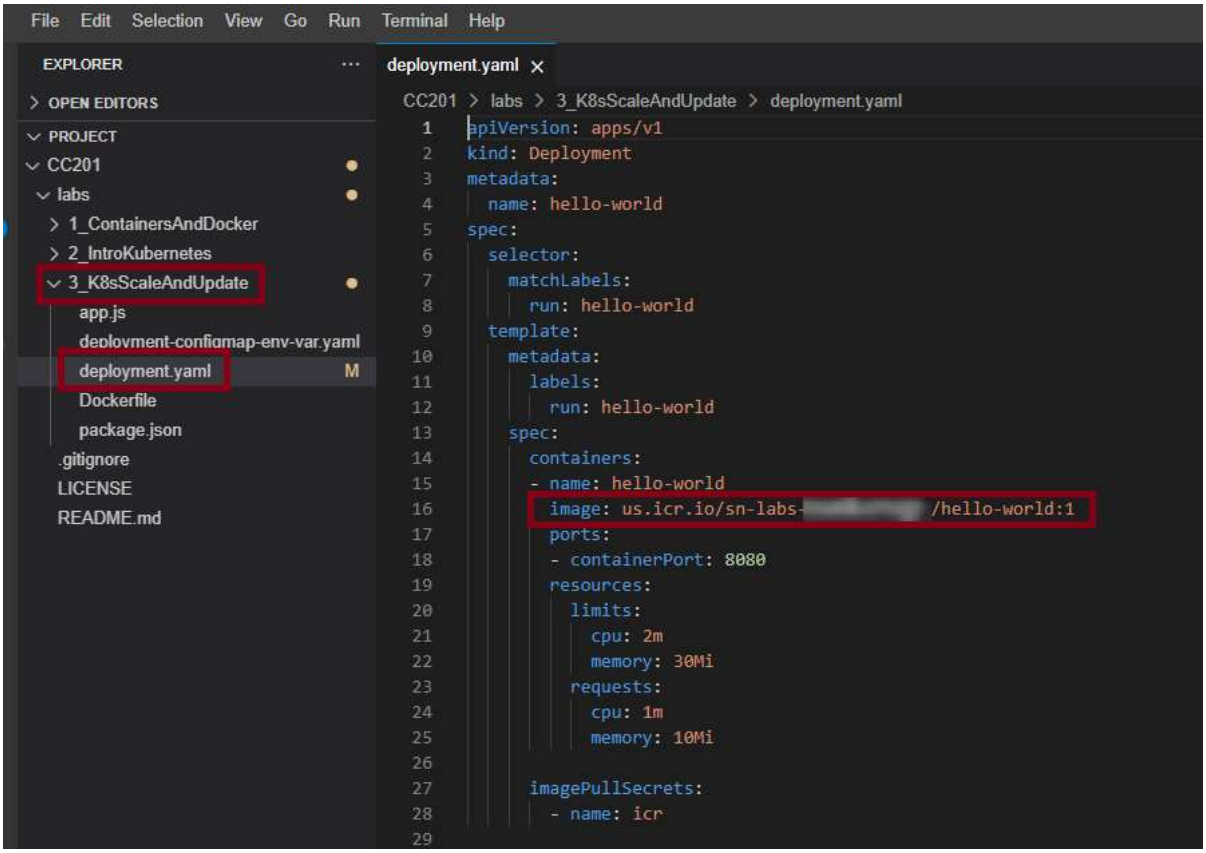
```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ docker build -t us.icr.io/$MY_NAMESPACE/hello-
Sending build context to Docker daemon  6.144kB
Step 1/6 : FROM node:9.4.0-alpine
9.4.0-alpine: Pulling from library/node
605ce1bd3f31: Pull complete
fe58b30348fe: Pull complete
46ef8987ccbd: Pull complete
Digest: sha256:9cd67a00ed111285460a83847720132204185e9321ec35dacec0d8b9bf674adf
Status: Downloaded newer image for node:9.4.0-alpine
 ---> b5f94997f35f
Step 2/6 : COPY app.js .
 ---> 2f029424b7dc
Step 3/6 : COPY package.json .
 ---> d4f6f041bcfa
Step 4/6 : RUN npm install &&    apk update &&    apk upgrade
 ---> Running in eb1b0f41cbd7
npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN hello-world-armada@0.0.1 No repository field.
npm WARN hello-world-armada@0.0.1 No license field.

added 50 packages in 1.708s
fetch http://dl-cdn.alpinelinux.org/alpine/v3.6/main/x86_64/APKINDEX.tar.gz
fetch http://dl-cdn.alpinelinux.org/alpine/v3.6/community/x86_64/APKINDEX.tar.gz
v3.6.5-44-gda55e27396 [http://dl-cdn.alpinelinux.org/alpine/v3.6/main]
v3.6.5-34-gf0ba0b43d5 [http://dl-cdn.alpinelinux.org/alpine/v3.6/community]
OK: 8448 distinct packages available
Upgrading critical system libraries and apk-tools:
(1/1) Upgrading apk-tools (2.7.5-r0 -> 2.7.6-r0)
Executing busybox-1.26.2-r9.trigger
Continuing the upgrade transaction with new apk-tools:
(1/7) Upgrading musl (1.1.16-r14 -> 1.1.16-r15)
(2/7) Upgrading busybox (1.26.2-r9 -> 1.26.2-r11)
Executing busybox-1.26.2-r11.post-upgrade
(3/7) Upgrading libressl2.5-libcrypto (2.5.5-r0 -> 2.5.5-r2)
(4/7) Upgrading libressl2.5-libssl (2.5.5-r0 -> 2.5.5-r2)
(5/7) Installing libressl2.5-libtls (2.5.5-r2)
(6/7) Installing ssl_client (1.26.2-r11)
(7/7) Upgrading musl-utils (1.1.16-r14 -> 1.1.16-r15)
Executing busybox-1.26.2-r11.trigger
OK: 5 MiB in 15 packages
Removing intermediate container eb1b0f41cbd7
 ---> 8064e924ec74
Step 5/6 : EXPOSE  8080
 ---> Running in 06b2f40f50c1
Removing intermediate container 06b2f40f50c1
 ---> 74d97beb1311
Step 6/6 : CMD node app.js
 ---> Running in 8388f224b326
Removing intermediate container 8388f224b326
 ---> ca395ff2f872
Successfully built ca395ff2f872
Successfully tagged us.icr.io/sn-labs-[redacted]/hello-world:1
The push refers to repository [us.icr.io/sn-labs-[redacted]/hello-world]
fc8314e02b47: Pushed
2e7bcf63d006: Pushed
609d2e4acfc9: Pushed
0804854a4553: Pushed
6bd4a62f5178: Pushed
9dfa40a0da3b: Pushed
1: digest: sha256:adb28bb0d3e133d2eb3563430dcd41a7a35eb816331430bb601c6a5375fe351b size: 1576
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

NOTE: If you have tried this lab earlier, there might be a possibility that the previous session is still persistent. In such case, you will see a **'Layer already Exists'** message instead of the **'Pushed'** message in the above output. We would recommend you to continue with the further steps of the lab.

Deploy the application to Kubernetes

1. Use the Explorer to edit `deployment.yaml` in this directory. The path to this file is `CC201/labs/3_K8sScaleAndUpdate/`. You need to insert your namespace where it says `<my_namespace>`. Make sure to save the file when you're done.

   NOTE: To know your namespace, run `echo $MY_NAMESPACE` in the terminal



2. Run your image as a Deployment.

```
1
kubectl apply -f deployment.yaml
```

Copied!

```
theia@theiadocker- :/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl apply -f deployment.yaml
deployment.apps/hello-world created
```

NOTE: If you have tried this lab earlier, there might be a possibility that the previous session is still persistent. In such a case, you will see an **'Unchanged'** message instead of the **'Created'** message in the above output. We would recommend you to continue with the further steps of the lab.

3. List Pods until the status is "Running".

```
1
kubectl get pods
```

Copied!

```
theia@theiadocker- :/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
hello-world-58985bb9fb-7nnqr  1/1     Running   0          4m52s
```

NOTE: Please move to the next step only after you see the pod status as **'Running'**. In case you see **'Container Creating'** as the output, please re-run the command in a few minutes.

4. In order to access the application, we have to expose it to the internet via a Kubernetes Service.

```
1
kubectl expose deployment/hello-world
```

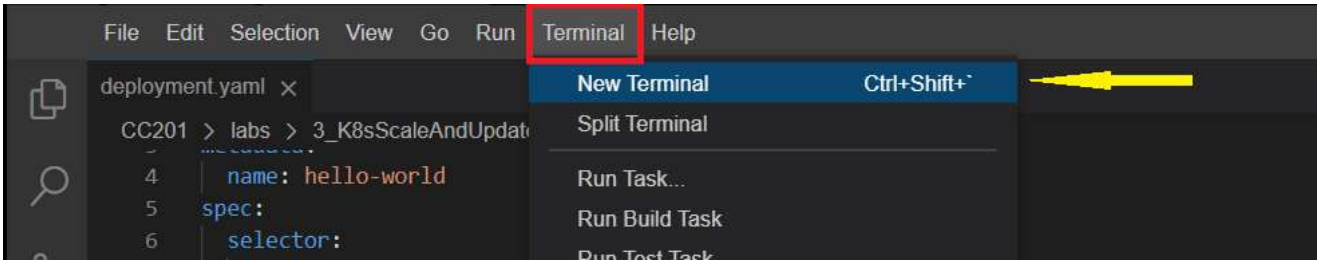
Copied!

This creates a service of type **ClusterIP**.

```
theia@theiadocker- :/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl expose deployment/hello-w
service/hello-world exposed
theia@theiadocker- :/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

5. Open a new terminal window using `Terminal > New Terminal`.

NOTE: Do not close the terminal window you were working on.



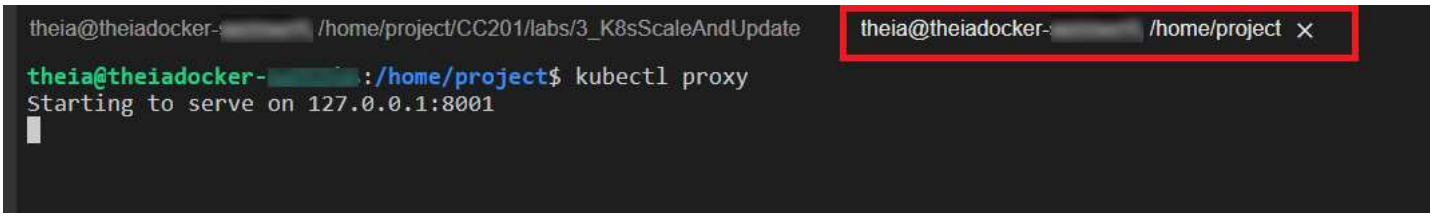
6. Cluster IPs are only accesible within the cluster. To make this externally accessible, we will create a proxy.

> **Note:** This is not how you would make an application externally accessible in a production scenario.

Run this command in the new terminal window since your environment variables need to be accessible in the original window for subsequent commands.

```
1
kubectl proxy
```

Copied!



This command will continue running until it exits. Keep it running so that you can continue to access your app.

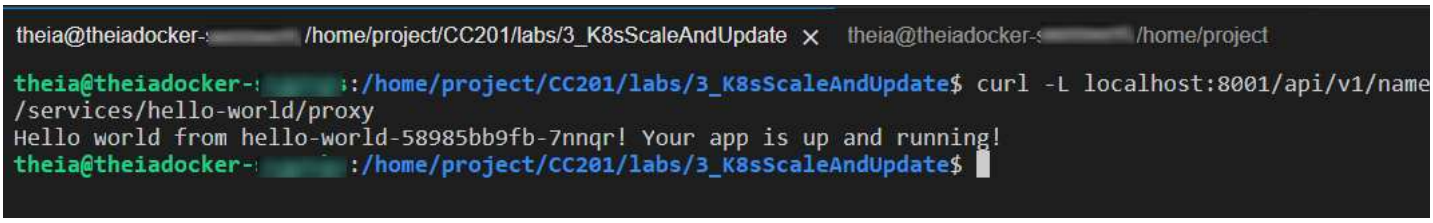
7. Go back to your original terminal window, ping the application to get a response.

> **NOTE:** Do not close the terminal window where the `proxy` command is still running.

```
1
curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy
```

Copied!

Observe the message "Hello world from hello-world-xxxxxxxx-xxxx. Your app is up and running!



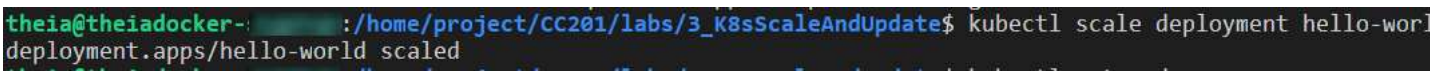
Scaling the application using a ReplicaSet

In real-world situations, load on an application can vary over time. If our application begins experiencing heightened load, we want to scale it up to accommodate that load. There is a simple `kubectl` command for scaling.

1. Use the `scale` command to scale up your Deployment. Make sure to run this in the terminal window that is not running the `proxy` command.

```
1
kubectl scale deployment hello-world --replicas=3
```

Copied!



2. Get Pods to ensure that there are now three Pods instead of just one. In addition, the status should eventually update to "Running" for all three.

```
1
kubectl get pods
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get pods
NAME                              READY   STATUS              RESTARTS   AGE
hello-world-58985bb9fb-7nnqr      1/1     Running             0          22m
hello-world-58985bb9fb-j9qkv      0/1     ContainerCreating   0          6s
hello-world-58985bb9fb-wg7nh      0/1     ContainerCreating   0          5s
```

3. As you did in the last lab, ping your application multiple times to ensure that Kubernetes is load-balancing across the replicas.

```
1
for i in `seq 10`; do curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy; done
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ for i in `seq 10`; do curl -L local
aces/sn-labs-$USERNAME/services/hello-world/proxy; done
Hello world from hello-world-58985bb9fb-7nnqr! Your app is up and running!
Hello world from hello-world-58985bb9fb-wg7nh! Your app is up and running!
Hello world from hello-world-58985bb9fb-wg7nh! Your app is up and running!
Hello world from hello-world-58985bb9fb-wg7nh! Your app is up and running!
Hello world from hello-world-58985bb9fb-7nnqr! Your app is up and running!
Hello world from hello-world-58985bb9fb-j9qkv! Your app is up and running!
Hello world from hello-world-58985bb9fb-wg7nh! Your app is up and running!
Hello world from hello-world-58985bb9fb-wg7nh! Your app is up and running!
Hello world from hello-world-58985bb9fb-7nnqr! Your app is up and running!
Hello world from hello-world-58985bb9fb-7nnqr! Your app is up and running!
```

You should see that the queries are going to different Pods because of the effect of load-balancing.

4. Similarly, you can use the `scale` command to scale down your Deployment.

```
1
kubectl scale deployment hello-world --replicas=1
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl scale deployment hello-worl
deployment.apps/hello-world scaled
```

5. Check the Pods to see that two are deleted or being deleted.

```
1
kubectl get pods
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get pods
NAME                              READY   STATUS        RESTARTS   AGE
hello-world-58985bb9fb-7nnqr      1/1     Running       0          23m
hello-world-58985bb9fb-j9qkv      1/1     Terminating   0          44s
hello-world-58985bb9fb-wg7nh      1/1     Terminating   0          43s
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

6. Please wait for some time & run the same command again to ensure that only one pod exists.

```
1
kubectl get pods
```

Copied!

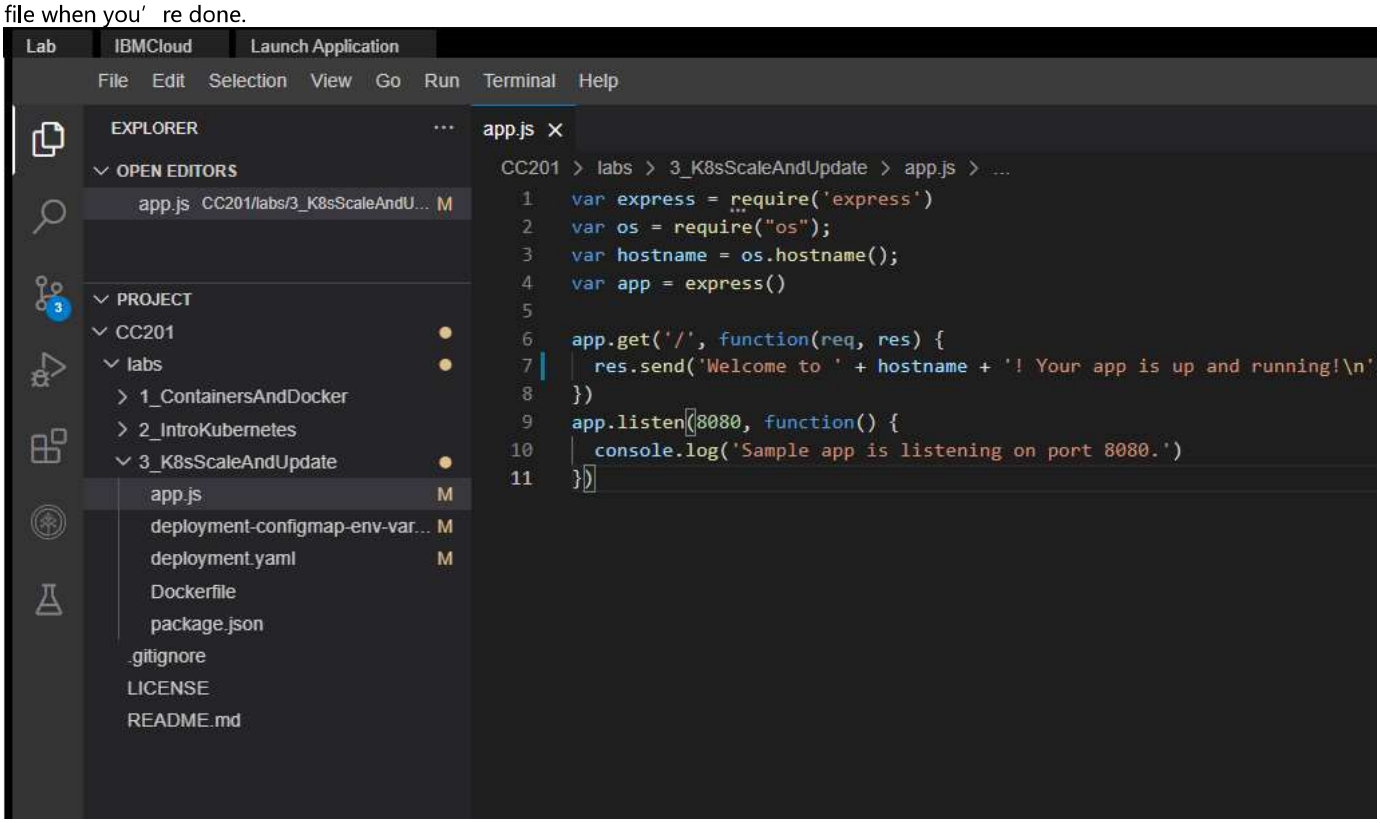
```
theia@theiadocker-[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get pods
NAME                              READY   STATUS    RESTARTS   AGE
hello-world-79c5684b95-5x4wp      1/1     Running   0          102s
theia@theiadocker-[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

Perform rolling updates

Rolling updates are an easy way to update our application in an automated and controlled fashion. To simulate an update, let’s first build a new version of our application and push it to Container Registry.

1. Use the Explorer to edit `app.js`. The path to this file is `CC201/labs/3_K8sScaleAndUpdate/`. Change the welcome message from `'Hello world from ' + hostname + '! Your app is up and running!\n'` to `'Welcome to ' + hostname + '! Your app is up and running!\n'`. Make sure to save the

file when you’re done.



2. Build and push this new version to Container Registry. Update the tag to indicate that this is a second version of this application. Make sure to use the terminal window that isn’t running the `proxy` command.

   NOTE: Do not close the terminal that is running the `proxy` command

```
1. l
1. docker build -t us.icr.io/$MY_NAMESPACE/hello-world:2 . && docker push us.icr.io/$MY_NAMESPACE/hello-world:2
```

Copied!

```
theia@theiadocker-:/home/project/CC201/labs/3_K8sScaleAndUpdate$ docker build -t us.icr.io/$MY_NAMESPACE/hello-
2 . && docker push us.icr.io/$MY_NAMESPACE/hello-world:2
Sending build context to Docker daemon  6.144kB
Step 1/6 : FROM node:9.4.0-alpine
 ---> b5f94997f35f
Step 2/6 : COPY app.js .
 ---> f25f279213f5
Step 3/6 : COPY package.json .
 ---> 7d7357f01482
Step 4/6 : RUN npm install &&    apk update &&    apk upgrade
 ---> Running in cf3918a57f10
npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN hello-world-armada@0.0.1 No repository field.
npm WARN hello-world-armada@0.0.1 No license field.

added 50 packages in 1.662s
fetch http://dl-cdn.alpinelinux.org/alpine/v3.6/main/x86_64/APKINDEX.tar.gz
fetch http://dl-cdn.alpinelinux.org/alpine/v3.6/community/x86_64/APKINDEX.tar.gz
v3.6.5-44-gda55e27396 [http://dl-cdn.alpinelinux.org/alpine/v3.6/main]
v3.6.5-34-gf0ba0b43d5 [http://dl-cdn.alpinelinux.org/alpine/v3.6/community]
OK: 8448 distinct packages available
Upgrading critical system libraries and apk-tools:
(1/1) Upgrading apk-tools (2.7.5-r0 -> 2.7.6-r0)
Executing busybox-1.26.2-r9.trigger
Continuing the upgrade transaction with new apk-tools:
(1/7) Upgrading musl (1.1.16-r14 -> 1.1.16-r15)
(2/7) Upgrading busybox (1.26.2-r9 -> 1.26.2-r11)
Executing busybox-1.26.2-r11.post-upgrade
(3/7) Upgrading libressl2.5-libcrypto (2.5.5-r0 -> 2.5.5-r2)
(4/7) Upgrading libressl2.5-libssl (2.5.5-r0 -> 2.5.5-r2)
(5/7) Installing libressl2.5-libtls (2.5.5-r2)
(6/7) Installing ssl_client (1.26.2-r11)
(7/7) Upgrading musl-utils (1.1.16-r14 -> 1.1.16-r15)
Executing busybox-1.26.2-r11.trigger
OK: 5 MiB in 15 packages
Removing intermediate container cf3918a57f10
 ---> 80a17e776942
Step 5/6 : EXPOSE  8080
 ---> Running in a868dd640957
Removing intermediate container a868dd640957
 ---> e2e4773f5ed3
Step 6/6 : CMD node app.js
 ---> Running in dad7dc244e00
Removing intermediate container dad7dc244e00
 ---> ce8704ad297f
Successfully built ce8704ad297f
Successfully tagged us.icr.io/sn-labs-[redacted]/hello-world:2
The push refers to repository [us.icr.io/sn-labs-[redacted]/hello-world]
237f3805cc80: Pushed
2e7bcf63d006: Layer already exists
ceb7ca869893: Pushed
0804854a4553: Layer already exists
6bd4a62f5178: Layer already exists
9dfa40a0da3b: Layer already exists
2: digest: sha256:839ba8ee302a5b4be4bcc4ad0c701b2c76627a592c9c7788f9e30674ab900748 size: 1576
theia@theiadocker-:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

3. List images in Container Registry to see all the different versions of this application that you have pushed so far.

```
1. l
1. ibmcloud cr images
```

Copied!

```
Problems    theia@theiadocker-snehar: /home/project/CC201/labs/3_K8sScaleAndUpdate ×    theia@theiadocker-snehar: /home/project

theia@theiadocker-snehar:/home/project/CC201/labs/3_K8sScaleAndUpdate$ ibmcloud cr images
Listing images...

Repository                                                          Tag      Digest        Namespace
us.icr.io/sn-labs-snehar/hello-world                                1        584c58ccc30f  sn-labs-s
us.icr.io/sn-labs-snehar/hello-world                                2        0cddeed2656c  sn-labs-s
us.icr.io/sn-labsassets/categories-watson-nlp-runtime               latest   6b01b1e5527b  sn-labsas
us.icr.io/sn-labsassets/classification-watson-nlp-runtime           latest   dbd407898549  sn-labsas
us.icr.io/sn-labsassets/concepts-watson-nlp-runtime                 latest   1e4741f10569  sn-labsas
us.icr.io/sn-labsassets/custom-watson-nlp-runtime                   latest   f6513e19a33d  sn-labsas
us.icr.io/sn-labsassets/detag-watson-nlp-runtime                    latest   38916c2119fc  sn-labsas
us.icr.io/sn-labsassets/emotion-watson-nlp-runtime                  latest   1c9de1d27318  sn-labsas
us.icr.io/sn-labsassets/entity-mentions-bert-watson-nlp-runtime     latest   57d92957214f  sn-labsas
us.icr.io/sn-labsassets/entity-mentions-bilstm-watson-nlp-runtime   latest   76dbd3bdb12b  sn-labsas
us.icr.io/sn-labsassets/entity-mentions-rbr-multi-watson-nlp-runtime latest  577399d7b4e7  sn-labsas
us.icr.io/sn-labsassets/entity-mentions-rbr-watson-nlp-runtime      latest   506cc92ecd3f  sn-labsas
us.icr.io/sn-labsassets/entity-mentions-sire-watson-nlp-runtime     latest   cd4e48efd3f6  sn-labsas
us.icr.io/sn-labsassets/entity-mentions-transformer-watson-nlp-runtime latest 0584c56563ce  sn-labsas
us.icr.io/sn-labsassets/instructions-splitter                       latest   2af122cfe4ee  sn-labsas
us.icr.io/sn-labsassets/keywords-watson-nlp-runtime                 latest   e2b9dc471ae0  sn-labsas
us.icr.io/sn-labsassets/lang-detect-watson-nlp-runtime              latest   4d3b44e72af0  sn-labsas
us.icr.io/sn-labsassets/noun-phrases-watson-nlp-runtime             latest   c696f6af9797  sn-labsas
us.icr.io/sn-labsassets/pgadmin-theia                               latest   0adf67ad81a3  sn-labsas
us.icr.io/sn-labsassets/phpmyadmin                                  latest   b66c30786353  sn-labsas
us.icr.io/sn-labsassets/relations-sire-watson-nlp-runtime           latest   65c2e74995d5  sn-labsas
us.icr.io/sn-labsassets/relations-transformer-watson-nlp-runtime    latest   18ffd6c35726  sn-labsas
us.icr.io/sn-labsassets/relations-watson-nlp-runtime                latest   3547dcc15c43  sn-labsas
```

Ensure that the new image shows `No Issues`, else re-run the image several times till there are no issues.

4. Update the deployment to use this version instead.

```
1
kubectl set image deployment/hello-world hello-world=us.icr.io/$MY_NAMESPACE/hello-world:2
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl set image deployment/hello-world hello
=us.icr.io/$MY_NAMESPACE/hello-world:2
deployment.apps/hello-world image updated
```

5. Get a status of the rolling update by using the following command:

```
1
kubectl rollout status deployment/hello-world
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl rollout status deployment/hello-world
deployment "hello-world" successfully rolled out
```

6. You can also get the Deployment with the `wide` option to see that the new tag is used for the image.

```
1
kubectl get deployments -o wide
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get deployments -o wide
NAME          READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS    IMAGES                                  SELECTOR
hello-world   1/1     1            1           39m   hello-world   us.icr.io/sn-labs-[redacted]/hello-world:2   run=hello
```

Look for the `IMAGES` column and ensure that the tag is `2`.

7. Ping your application to ensure that the new welcome message is displayed.

```
1
curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ curl -L localhost:8001/api/v1/namespaces/sn-la
ERNAME/services/hello-world/proxy
Welcome to hello-world-5cc6f44c5-zhh96! Your app is up and running!
```

8. It’s possible that a new version of an application contains a bug. In that case, Kubernetes can roll back the Deployment like this:

```
1
```

```
1. kubectl rollout undo deployment/hello-world
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl rollout undo deployment/hello-world
deployment.apps/hello-world rolled back
```

9. Get a status of the rolling update by using the following command:

```
1. 1
1. kubectl rollout status deployment/hello-world
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl rollout status deployment/hello-world
deployment "hello-world" successfully rolled out
```

10. Get the Deployment with the `wide` option to see that the old tag is used.

```
1. 1
1. kubectl get deployments -o wide
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get deployments -o wide
NAME          READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS    IMAGES                                        SELECTOR
hello-world   1/1     1            1           40m   hello-world   us.icr.io/sn-labs-[redacted]/hello-world:1    run=hello
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

Look for the `IMAGES` column and ensure that the tag is `1`.

11. Ping your application to ensure that the earlier **'Hello World..Your app is up & running!'** message is displayed.

```
1. 1
1. curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy
```

Copied!

```
theia@theiadocker-[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$ curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNA
Hello world from hello-world-79c5684b95-6xr4l! Your app is up and running!
theia@theiadocker[redacted]home/project/CC201/labs/3_K8sScaleAndUpdate$
```

Using a ConfigMap to store configuration

ConfigMaps and Secrets are used to store configuration information separate from the code so that nothing is hardcoded. It also lets the application pick up configuration changes without needing to be redeployed. To demonstrate this, we'll store the application's message in a ConfigMap so that the message can be updated simply by updating the ConfigMap.

1. Create a ConfigMap that contains a new message.

```
1. 1
1. kubectl create configmap app-config --from-literal=MESSAGE="This message came from a ConfigMap!"
```

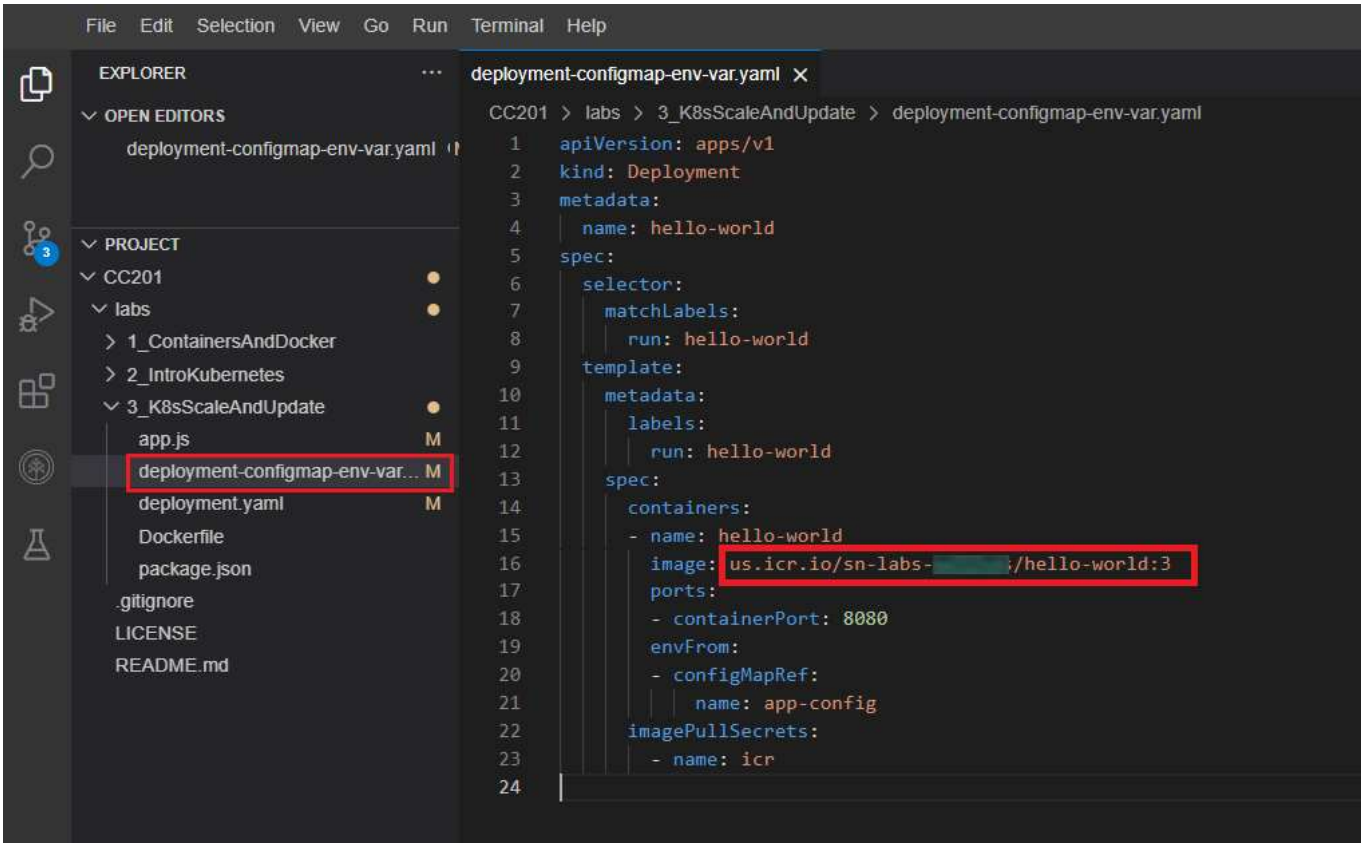
Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl create configmap app-config --from-lit
ESSAGE="This message came from a ConfigMap!"
configmap/app-config created
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

> **NOTE:** If you have tried this lab earlier, there might be a possibility that the previous session is still persistent. In such a case, you will see an **'error: failed to create configmap: configmaps "app-config" already exists'** message, instead of the **'Created'** message as below. We would recommend you to continue with the further steps of the lab.

```
theia@theiadocker[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl create configmap app-config --from-literal=MESSA
igMap!"
error: failed to create configmap: configmaps "app-config" already exists
theia@theiadocker-[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

2. Use the Explorer to edit `deployment-configmap-env-var.yaml`. The path to this file is `CC201/labs/3_K8sScaleAndUpdate/`. You need to insert your namespace where it says `<my_namespace>`. Make sure to save the file when you're done.



3. In the same file, notice the section reproduced below. The bottom portion indicates that environment variables should be defined in the container from the data in a ConfigMap named `app-config`.

```
1
2
3
4
5
6
7
8
```

```
containers:
- name: hello-world
  image: us.icr.io/<my_namespace>/hello-world:3
  ports:
  - containerPort: 8080
  envFrom:
  - configMapRef:
    name: app-config
```

Copied!

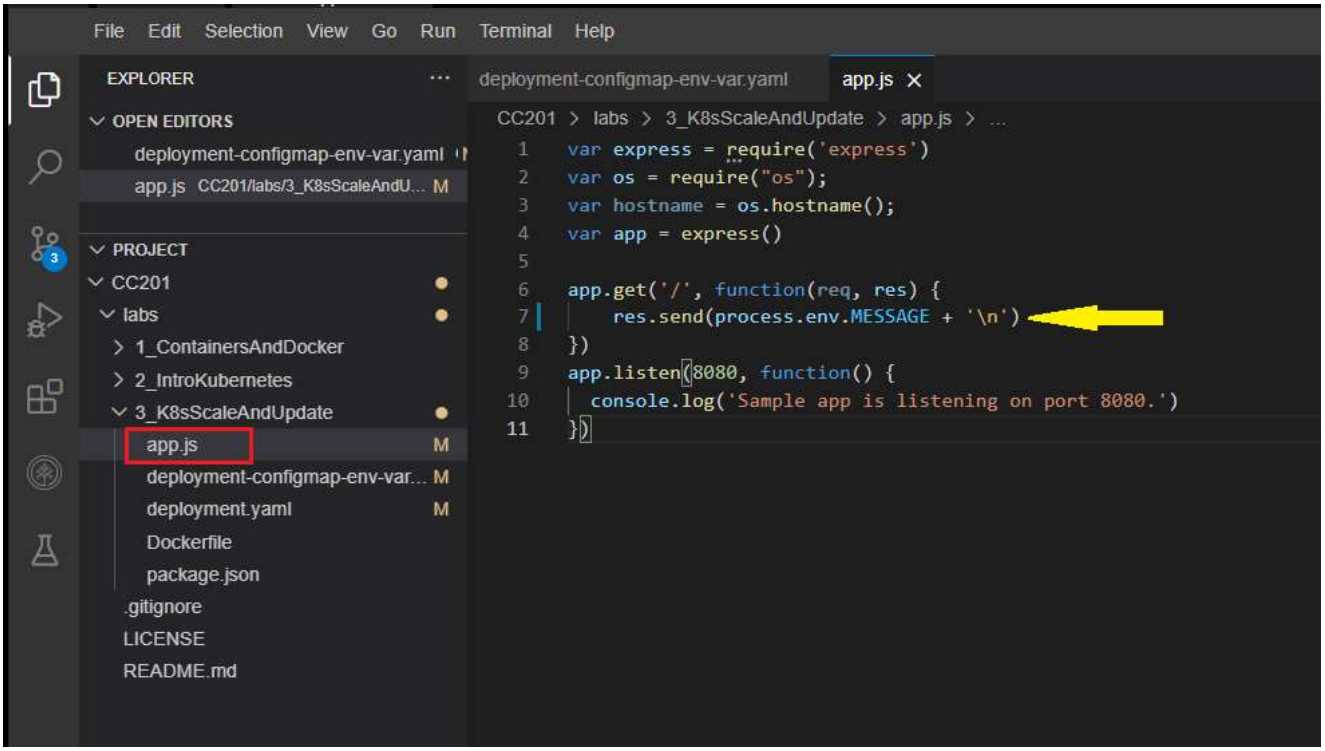
4. Use the Explorer to open the `app.js` file. The path to this file is `CC201/labs/3_K8sScaleAndUpdate/`. Find the line that says, `res.send('Welcome to ' + hostname + '! Your app is up and running!\n')`.

Edit this line to look like the following:

```
1
```

```
res.send(process.env.MESSAGE + '\n')
```

Copied!



Make sure to save the file when you're done. This change indicates that requests to the app will return the environment variable `MESSAGE`.

5. Build and push a new image that contains your new application code.

```
1
docker build -t us.icr.io/$MY_NAMESPACE/hello-world:3 . && docker push us.icr.io/$MY_NAMESPACE/hello-world:3
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ docker build -t us.icr.io/$MY_NAMESPACE/hello-
3 . && docker push us.icr.io/$MY_NAMESPACE/hello-world:3
Sending build context to Docker daemon  6.144kB
Step 1/6 : FROM node:9.4.0-alpine
 ---> b5f94997f35f
Step 2/6 : COPY app.js .
 ---> 3f0b66f4e16f
Step 3/6 : COPY package.json .
 ---> 8bcec318978a
Step 4/6 : RUN npm install &&    apk update &&    apk upgrade
 ---> Running in 7d432320817c
npm notice created a lockfile as package-lock.json. You should commit this file.
npm WARN hello-world-armada@0.0.1 No repository field.
npm WARN hello-world-armada@0.0.1 No license field.

added 50 packages in 1.615s
fetch http://dl-cdn.alpinelinux.org/alpine/v3.6/main/x86_64/APKINDEX.tar.gz
fetch http://dl-cdn.alpinelinux.org/alpine/v3.6/community/x86_64/APKINDEX.tar.gz
v3.6.5-44-gda55e27396 [http://dl-cdn.alpinelinux.org/alpine/v3.6/main]
v3.6.5-34-gf0ba0b43d5 [http://dl-cdn.alpinelinux.org/alpine/v3.6/community]
OK: 8448 distinct packages available
Upgrading critical system libraries and apk-tools:
(1/1) Upgrading apk-tools (2.7.5-r0 -> 2.7.6-r0)
Executing busybox-1.26.2-r9.trigger
Continuing the upgrade transaction with new apk-tools:
(1/7) Upgrading musl (1.1.16-r14 -> 1.1.16-r15)
(2/7) Upgrading busybox (1.26.2-r9 -> 1.26.2-r11)
Executing busybox-1.26.2-r11.post-upgrade
(3/7) Upgrading libressl2.5-libcrypto (2.5.5-r0 -> 2.5.5-r2)
(4/7) Upgrading libressl2.5-libssl (2.5.5-r0 -> 2.5.5-r2)
(5/7) Installing libressl2.5-libtls (2.5.5-r2)
(6/7) Installing ssl_client (1.26.2-r11)
(7/7) Upgrading musl-utils (1.1.16-r14 -> 1.1.16-r15)
Executing busybox-1.26.2-r11.trigger
OK: 5 MiB in 15 packages
Removing intermediate container 7d432320817c
 ---> ed77983749d5
Step 5/6 : EXPOSE  8080
 ---> Running in 5686c39353f8
Removing intermediate container 5686c39353f8
 ---> 529399efa32f
Step 6/6 : CMD node app.js
 ---> Running in 942b22038f71
Removing intermediate container 942b22038f71
 ---> 6e2bc34c6c21
Successfully built 6e2bc34c6c21
Successfully tagged us.icr.io/sn-labs-[redacted]/hello-world:3
The push refers to repository [us.icr.io/sn-labs-[redacted]/hello-world]
d4bcd81b0ba6: Pushed
2e7bcf63d006: Layer already exists
adf91d207735: Pushed
0804854a4553: Layer already exists
6bd4a62f5178: Layer already exists
9dfa40a0da3b: Layer already exists
3: digest: sha256:b9b9ee39218a0bc88a121fa60e6a1d1d4a5c5eae2d6122fc87b8d7f3911e5a8f size: 1576
```

The `deployment-configmap-env-var.yaml` file is already configured to use the tag `3`.

6. Apply the new Deployment configuration.

```
1. l
1. kubectl apply -f deployment-configmap-env-var.yaml
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl apply -f deployment-configmap-env-var.
deployment.apps/hello-world configured
```

7. Ping your application again to see if the message from the environment variable is returned.

   NOTE: You can run this command again. As it may not show the `"This message came from a ConfigMap!"` message right away.

```
1. l
1. curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ curl -L localhost:8001/api/v1/namespaces/sn-la
ERNAME/services/hello-world/proxy

This message came from a ConfigMap!
```

If you see the message, "This message came from a ConfigMap!", then great job!

> **NOTE:** If your previous session is still persisting, you might see the below output. If so, we would recommend you to move to the further steps of the lab.

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNA
This message is different, and you didn't have to rebuild the image!
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

8. Because the configuration is separate from the code, the message can be changed without rebuilding the image. Using the following command, delete the old ConfigMap and create a new one with the same name but a different message.

```
1. 1
1. kubectl delete configmap app-config && kubectl create configmap app-config --from-literal=MESSAGE="This message is different, and you didn't have t
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl delete configmap app-config && kubectl
e configmap app-config --from-literal=MESSAGE="This message is different, and you didn't have to rebuild the image!"
configmap "app-config" deleted
configmap/app-config created
```

9. Restart the Deployment so that the containers restart. This is necessary since the environment variables are set at start time.

```
1. 1
1. kubectl rollout restart deployment hello-world
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl rollout restart deployment hello-world
deployment.apps/hello-world restarted
```

10. Ping your application again to see if the new message from the environment variable is returned.

```
1. 1
1. curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ curl -L localhost:8001/api/v1/namespaces/sn-la
ERNAME/services/hello-world/proxy
This message is different, and you didn't have to rebuild the image!
```

Autoscale the `hello-world` application using Horizontal Pod Autoscaler

1. Please add the following section to the `deployment.yaml` file under the `template.spec.containers` section for increasing the CPU resource utilization

```
1. 1
2. 2
3. 3
4. 4
5. 5
6. 6
```

```
1.          name: http
2.        resources:
3.          limits:
4.            cpu: 50m
5.          requests:
6.            cpu: 20m
```

Copied!

> **Note:** After making the changes, do not forget to save the file.

The updated file will be as below:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
spec:
  selector:
    matchLabels:
      run: hello-world
  template:
    metadata:
      labels:
        run: hello-world
    spec:
      containers:
      - image: us.icr.io/sn-labs-[redacted]/hello-world:1
        imagePullPolicy: Always
        name: hello-world
        ports:
        - containerPort: 8080
          name: http
        resources:
          limits:
            cpu: 50m
          requests:
            cpu: 20m
      imagePullSecrets:
        - name: icr
```

2. Apply the deployment:

```
1
kubectl apply -f deployment.yaml
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl apply -f deployment.yaml
deployment.apps/hello-world configured
```

3. Autoscale the `hello-world` deployment using the below command:

```
1
kubectl autoscale deployment hello-world --cpu-percent=5 --min=1 --max=10
```

Copied!

```
theia@theiadocker-[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl autoscale deployment hello-world --cpu-percent=
horizontalpodautoscaler.autoscaling/hello-world autoscaled
theia@theiadocker[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

4. You can check the current status of the newly-made HorizontalPodAutoscaler, by running:

```
1
kubectl get hpa hello-world
```

Copied!

```
^Ctheia@theiadocker-[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get hpa hello-world
NAME          REFERENCE                TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
hello-world   Deployment/hello-world   0%/5%     1         10        1          19m
theia@theiadocker[redacted]/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

5. Please ensure that the kubernetes proxy is still running in the 2nd terminal. If it is not, please start it again by running:

```
1
kubectl proxy
```

Copied!

6. Open another new terminal and enter the below command to spam the app with multiple requests for increasing the load:

```
1
for i in `seq 100000`; do curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/services/hello-world/proxy; done
```

Copied!

```
theia@theiadocker-[redacted]:/home/project$ for i in `seq 100000`; do curl -L localhost:8001/api/v1/namespaces/sn-labs-$USERNAME/
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
This message is different, and you didn't have to rebuild the image!
```

Continue further commands in the 1st terminal

7. Run the below command to observe the replicas increase in accordance with the autoscaling:

```
1. 1
1. kubectl get hpa hello-world --watch
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get hpa hello-world --watch
NAME          REFERENCE                TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
hello-world   Deployment/hello-world   0%/5%     1         10        1          153m
hello-world   Deployment/hello-world   30%/5%    1         10        1          154m
hello-world   Deployment/hello-world   30%/5%    1         10        4          154m
hello-world   Deployment/hello-world   30%/5%    1         10        6          154m
hello-world   Deployment/hello-world   25%/5%    1         10        6          154m
hello-world   Deployment/hello-world   17%/5%    1         10        6          155m
hello-world   Deployment/hello-world   17%/5%    1         10        7          155m
hello-world   Deployment/hello-world   22%/5%    1         10        7          156m
hello-world   Deployment/hello-world   22%/5%    1         10        9          156m
```

You will see an increase in the number of replicas which shows that your application has been autoscaled.

Stop this command by pressing `CTRL + C`.

8. Run the below command to observe the details of the horizontal pod autoscaler:

```
1. 1
1. kubectl get hpa hello-world
```

Copied!

```
^Ctheia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl get hpa hello-world
NAME          REFERENCE                TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
hello-world   Deployment/hello-world   5%/5%     1         10        9          160m
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

You will notice that the number of replicas has increased now.

9. Stop the proxy and the load generation commands running in the other 2 terminal by pressing `CTRL + C`.
10. Delete the Deployment.

```
1. 1
1. kubectl delete deployment hello-world
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl delete  deployment hello-world
deployment.apps "hello-world" deleted
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

11. Delete the Service.

```
1. 1
1. kubectl delete service hello-world
```

Copied!

```
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$ kubectl delete service hello-world
service "hello-world" deleted
theia@theiadocker-[redacted]:/home/project/CC201/labs/3_K8sScaleAndUpdate$
```

Congratulations! You have completed the lab for the third module of this course.

Changelog

Date	Version	Changed by	Change Description
2022-04-07	1.1	Samaah Sarang	Updated Lab instructions & images
2022-04-13	1.2	Samaah Sarang	Updated Lab instructions
2022-04-14	1.3	K Sundararajan	Updated Lab instructions & images
2022-04-18	1.4	K Sundararajan	Updated Lab instructions
2022-04-19	1.5	K Sundararajan	Updated Lab instructions
2022-07-25	1.6	K Sundararajan	Updated Lab instructionsto include HPA uisng kubectl
2022-08-02	1.7	K Sundararajan	Added new IDSN logo
2022-08-12	1.8	K Sundararajan	Updated Lab instructions
2022-08-16	1.9	K Sundararajan	Updated a screenshot
2023-04-21	2.0	Sneha R Baddi	Screenshot updated

© IBM Corporation 2022. All rights reserved.